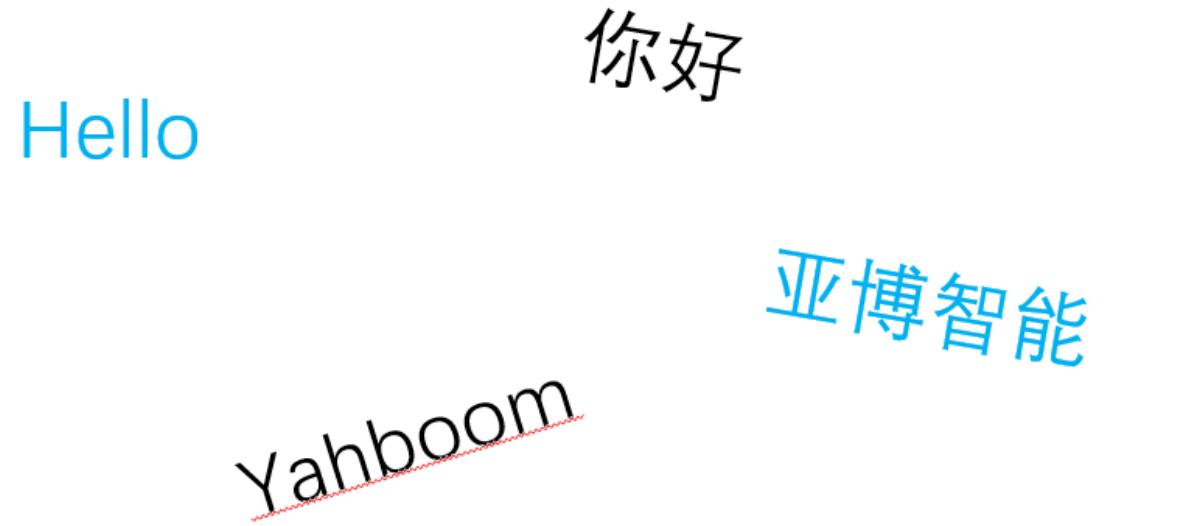
OCR 字符识别

OCR 字符识别
- 例程实验效果
- 代码讲解
  - 代码结构
  - 部分代码

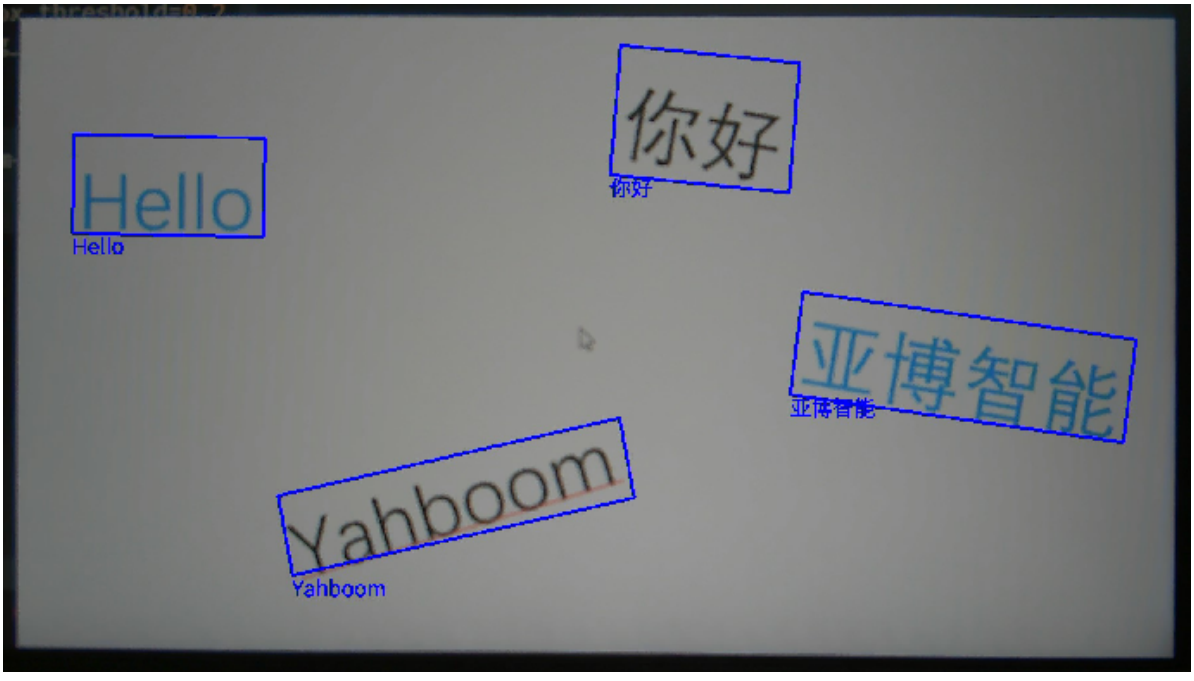
例程实验效果

在上一节OCR检测的基础上，我们再来实现一下简单的识别功能

运行这一节的例程代码，对准我们上一节准备好的图片：



K230识别的效果如下（支持中文和英文）：



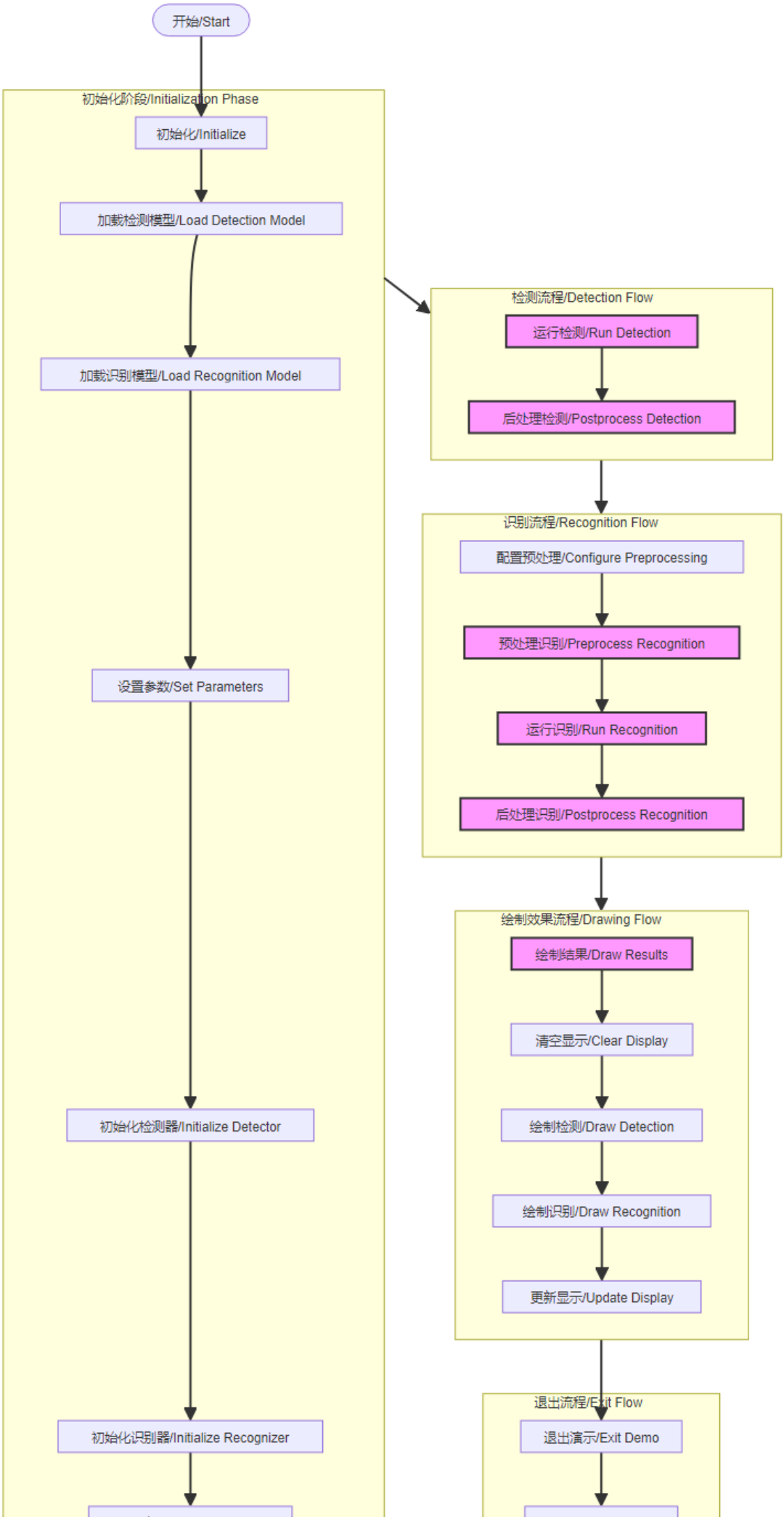
代码讲解

我们来看一下代码是如何实现的，先来看看总体的代码结构

代码结构

1. 初始化阶段/Initialization Phase:
    - 加载检测模型/Load detection model
    - 加载识别模型/Load recognition model
    - 设置各种参数/Set parameters
    - 初始化检测器/Initialize detector
    - 初始化识别器/Initialize recognizer
    - 读取字典/Read dictionary
2. 检测流程/Detection Flow:
    - 运行检测/Run detection
    - 后处理检测结果/Postprocess detection
3. 识别流程/Recognition Flow:
    - 配置预处理/Configure preprocessing
    - 预处理识别输入/Preprocess recognition input
    - 运行识别模型/Run recognition
    - 后处理识别输出/Postprocess recognition output
4. 绘制效果流程/Drawing Flow:
    - 清空显示/Clear display
    - 绘制检测结果/Draw detection
    - 绘制识别结果/Draw recognition
    - 更新显示/Update display
5. 退出流程/Exit Flow:
    - 退出演示/Exit demo
    - 清理资源/Clean up

流程图如下：


开始/Start
初始化阶段/Initialization Phase
初始化/Initialize
加载检测模型/Load Detection Model
加载识别模型/Load Recognition Model
设置参数/Set Parameters
初始化检测器/Initialize Detector
初始化识别器/Initialize Recognizer
检测流程/Detection Flow
运行检测/Run Detection
后处理检测/Postprocess Detection
识别流程/Recognition Flow
配置预处理/Configure Preprocessing
预处理识别/Preprocess Recognition
运行识别/Run Recognition
后处理识别/Postprocess Recognition
绘制效果流程/Drawing Flow
绘制结果/Draw Results
清空显示/Clear Display
绘制检测/Draw Detection
绘制识别/Draw Recognition
更新显示/Update Display
退出流程/Exit Flow
退出演示/Exit Demo



部分代码

完整代码请参考文件【源码汇总 / 09.Scene / 02.ocr_rec.py】

```
    # run函数，执行推理
    # Run function for inference
    def run(self, input_np):
        # 先进行OCR检测 / First perform OCR detection
        det_res = self.ocr_det.run(input_np)
        boxes = []
        ocr_res = []
        for det in det_res:
            # 对得到的每个检测框执行OCR识别 / Perform OCR recognition on each
detected box
            self.ocr_rec.config_preprocess(input_image_size=[det[0].shape[2],
det[0].shape[1]], input_np=det[0])
            ocr_str = self.ocr_rec.run(det[0])
            ocr_res.append(ocr_str)
            boxes.append(det[1])
            # 执行垃圾回收，减少内存占用 / Perform garbage collection to reduce
memory usage
            gc.collect()
        return boxes, ocr_res

    # 绘制OCR检测识别效果
    # Draw OCR detection and recognition results
    def draw_result(self, pl, det_res, rec_res):
        # 清除叠加层 / Clear overlay layer
        pl.osd_img.clear()
        if det_res:
            # 循环绘制所有检测到的框 / Loop through all detected boxes
            for j in range(len(det_res)):
                # 将原图的坐标点转换成显示的坐标点，循环绘制四条直线，得到一个矩形框
                # Convert coordinates from original image to display coordinates
                # Draw four lines to form a rectangle
                for i in range(4):
                    # 坐标转换 / Coordinate conversion
                    x1 = det_res[j][(i * 2)] / self.rgb888p_size[0] *
self.display_size[0]
                    y1 = det_res[j][(i * 2 + 1)] / self.rgb888p_size[1] *
self.display_size[1]
                    x2 = det_res[j][((i + 1) * 2) % 8] / self.rgb888p_size[0] *
self.display_size[0]
                    y2 = det_res[j][((i + 1) * 2 + 1) % 8] /
self.rgb888p_size[1] * self.display_size[1]
                    # 绘制线段 / Draw line segment
                    pl.osd_img.draw_line((int(x1), int(y1), int(x2), int(y2)),
color=(255, 0, 0, 255), thickness=5)
                # 在框上方绘制识别文本 / Draw recognized text above the box
                pl.osd_img.draw_string_advanced(int(x1), int(y1), 32,
rec_res[j], color=(0, 0, 255))
```